

SkillBridge — Full Deployment Guide

Frontend (Vercel) • Payments API (Railway) • Data & Auth (Supabase) • Stripe

This document explains how to deploy the SkillBridge web application, host the Stripe payment backend, and manage environment variables when working in a team. **Secrets never belong in Git.**

Table of contents

1. [Architecture overview](#)
 2. [Git, `.env`, and collaborating safely](#)
 3. [Environment variables reference](#)
 4. [Deploy the payment API on Railway](#)
 5. [Deploy the frontend on Vercel](#)
 6. [Supabase setup \(reminder\)](#)
 7. [Connecting all three systems](#)
 8. [Local development](#)
 9. [Test mode vs live mode](#)
 10. [The `railway.toml` file](#)
 11. [Troubleshooting](#)
 12. [Exporting this guide to PDF](#)
-

1. Architecture overview

Component	Role	Typical host
React app	User interface, Stripe.js in the browser	Vercel
Supabase	Database, authentication, app data	Supabase Cloud (managed)

Component	Role	Typical host
Payment API	Holds <code>STRIPE_SECRET_KEY</code> , creates PaymentIntents, CORS for your frontend	Railway (Node process)
Stripe	Card processing, dashboards, webhooks (optional later)	Stripe

Flow (simplified):

1. User opens the app from **Vercel**.
2. The app uses **Supabase** (URL + anon key) for login and data.
3. When paying, the browser calls **your Railway URL** (`/api/payments/...`).
4. Railway uses **Stripe secret key** to talk to Stripe and returns JSON (e.g. `clientSecret`).
5. Stripe Elements on the frontend completes the payment with Stripe.

Important: The **publishable** key (`pk_...`) can be in the frontend. The **secret** key (`sk_...`) must **only** run on the server (Railway), never in the browser and never committed to Git.

2. Git, `.env`, and collaborating safely

2.1 What must NOT be in Git

- `.env` (project root)
- `server/.env`
- `SkillBApp/.env`
- Any file containing real `sk_live_`, `sk_test_`, Supabase **service role** keys, or database passwords

These paths are listed in `.gitignore` on purpose.

2.2 What SHOULD be in Git

- `.env.example` — template for the React app variables (no real secrets).
- `server/.env.example` — template for the payment server (no real secrets).
- `railway.toml` — Railway build/start instructions (no secrets).
- Application source code and `vercel.json` as needed.

2.3 What each teammate does after `git clone`

1. Install dependencies: `npm install`
2. Copy templates:
 - Copy `.env.example` → `.env`
 - Copy `server/.env.example` → `server/.env`
3. Fill in **local** values (from Stripe dashboard, Supabase project settings, or a team password manager).
4. **Never** `git add` or commit `.env` / `server/.env`.

2.4 What production uses instead of committed `.env`

- **Vercel:** Project → **Settings** → **Environment Variables**
- **Railway:** Service → **Variables**
- **Supabase:** Keys come from the Supabase dashboard; only the **anon** key goes in the frontend.

No deploy platform reads your private `.env` from Git for production secrets.

3. Environment variables reference

3.1 Vercel (frontend — Create React App)

CRA bakes `REACT_APP_*` into the build at **build time**. If you change these, **redeploy** Vercel.

Variable	Description
<code>REACT_APP_SUPABASE_URL</code>	Supabase project URL (Settings → API).
<code>REACT_APP_SUPABASE_ANON_KEY</code>	Supabase anon public key (not the service role key).
<code>REACT_APP_STRIPE_PUBLISHABLE_KEY</code>	<code>pk_test_...</code> for testing, <code>pk_live_...</code> for production.
<code>REACT_APP_PAYMENT_API_URL</code>	Base URL for payment routes, must end with <code>/api/payments</code> . Example: <code>https://your-service.up.railway.app/api/payments</code>

No trailing slash after `payments` is required; the app appends `/create-intent`, etc.

3.2 Railway (payment API — `server/payment-api.js`)

Variable	Description
<code>STRIPE_SECRET_KEY</code>	<code>sk_test_...</code> or <code>sk_live_...</code> from Stripe Dashboard → API keys. Required. Without it the process crashes on startup.
<code>PORT</code>	Often set automatically by Railway. The app uses <code>process.env.PORT 3001</code> .
<code>ALLOWED_ORIGINS</code>	Comma-separated list of frontend origins, e.g. <code>https://your-app.vercel.app,https://your-preview.vercel.app</code>
<code>STRIPE_CORS_STRICT</code>	Set to <code>1</code> to enforce CORS against <code>ALLOWED_ORIGINS</code> plus built-in dev origins. Recommended for production.
<code>HOST</code>	Optional; default binds to <code>0.0.0.0</code> in code for container compatibility.

3.3 Local-only files (not in Git)

Root `.env` (for `npm start`): same names as Vercel `REACT_APP_*` where applicable.

`server/.env` : `STRIPE_SECRET_KEY`, optional `PORT`, `ALLOWED_ORIGINS`.

4. Deploy the payment API on Railway

4.1 Prerequisites

- GitHub repository contains:
 - `server/payment-api.js`
 - `package.json` with script `"payment-api": "node server/payment-api.js"`
 - `railway.toml` at the repository root (so Railway does **not** run `npm run build` for the React app).

4.2 Create the Railway project

1. Go to <https://railway.app> and sign in.
2. **New Project** → **Deploy from GitHub** → authorize and select your repository.
3. Railway creates a service linked to that repo.

4.3 How `railway.toml` helps

The file at the repo root tells Railway:

- **Build:** `npm install` (installs dependencies including `express`, `stripe`, `cors`, `dotenv`).
- **Start:** `npm run payment-api` → runs `node server/payment-api.js`.
- **Health check path:** `/health` for monitoring.

Without this, Railway might default to `npm run build`, which runs the React linter/build and **fails** on ESLint warnings in CI.

4.4 Set environment variables in Railway

1. Open your **service** → **Variables**.
2. Add at minimum:
 - `STRIPE_SECRET_KEY` = your Stripe secret key.
3. For production-style CORS:
 - `ALLOWED_ORIGINS` = `https://your-production-app.vercel.app`
 - `STRIPE_CORS_STRICT` = `1`
4. Save; Railway redeploys or trigger **Redeploy** manually.

4.5 Public URL and health check

1. **Settings** → **Networking** → **Generate Domain** (if not already).
2. Note the HTTPS URL, e.g. `https://skillbridge-payment-production.up.railway.app`.
3. Open in a browser:
`https://<your-railway-host>/health`
Expected JSON: `{"status":"ok","provider":"stripe"}`.

4.6 Verify payment routes exist

Base path for the Express app: `/api/payments`.

Examples:

- `POST /api/payments/create-intent`
- `POST /api/payments/confirm-intent`
- `GET /api/payments/status/:paymentIntentId`

The frontend uses `REACT_APP_PAYMENT_API_URL` pointing to the **base** `.../api/payments`.

5. Deploy the frontend on Vercel

5.1 Import project

1. Go to <https://vercel.com> → **Add New** → **Project**.
2. Import the **same** Git repository.
3. Framework preset: **Create React App** (or auto-detected).
4. **Build Command:** `npm run build`
5. **Output Directory:** `build`

5.2 Environment variables

In **Project** → **Settings** → **Environment Variables**, add (for **Production** and optionally **Preview**):

Name	Value
<code>REACT_APP_SUPABASE_URL</code>	From Supabase dashboard
<code>REACT_APP_SUPABASE_ANON_KEY</code>	Supabase anon key
<code>REACT_APP_STRIPE_PUBLISHABLE_KEY</code>	<code>pk_test_...</code> or <code>pk_live_...</code>
<code>REACT_APP_PAYMENT_API_URL</code>	<code>https://<railway-host>/api/payments</code>

5.3 Deploy and rebuild rule

Click **Deploy**. After **any** change to `REACT_APP_*`, trigger a **new deployment** so the bundle is rebuilt with the new values.

5.4 `vercel.json` (if present)

Your repo may include `vercel.json` with SPA rewrites so client-side routes work. No Stripe secrets belong there.

6. Supabase setup (reminder)

1. Create a project at <https://supabase.com>.
2. **Project Settings** → **API**: copy **Project URL** and **anon public** key.

3. Put those two into Vercel env vars and local `.env` as `REACT_APP_SUPABASE_URL` and `REACT_APP_SUPABASE_ANON_KEY`.
 4. Run SQL migrations / table setup from your repo docs as needed (separate from this guide).
-

7. Connecting all three systems

Checklist:

- ☐ Railway `/health` returns OK.
- ☐ Vercel `REACT_APP_PAYMENT_API_URL` exactly matches `https://<railway>/api/payments`.
- ☐ Stripe test **publishable** on Vercel matches test **secret** on Railway (both test mode).
- ☐ `ALLOWED_ORIGINS` on Railway includes your exact Vercel URL (`https://...`, no typo).
- ☐ Supabase URL and anon key on Vercel match your Supabase project.

Data flow:

```
Browser (Vercel-hosted app)
  → Supabase (auth + DB)
  → HTTPS → Railway /api/payments/*
    → Stripe API (secret key on Railway only)
```

8. Local development

8.1 One-time setup

```
npm install
copy .env.example .env
copy server\.env.example server\.env
```

Edit `.env` and `server/.env` with real test keys.

8.2 Two terminals

Terminal A — payment API

```
npm run payment-api
```

Terminal B — React app

```
npm start
```

8.3 Local payment URL

Typically:

- `REACT_APP_PAYMENT_API_URL=http://localhost:3001/api/payments`

in root `.env`, or rely on project defaults documented in `stripeService.js` for development.

8.4 Test card

Use Stripe test card `4242 4242 4242 4242`, any future expiry, any CVC, in **test** mode.

9. Test mode vs live mode

Mode	Publishable key	Secret key	Cards
Test	<code>pk_test_...</code>	<code>sk_test_...</code>	Test card numbers only
Live	<code>pk_live_...</code>	<code>sk_live_...</code>	Real cards; real money

Update **both** Vercel and Railway together when switching modes. Redeploy both after changing keys.

10. The `railway.toml` file

Location: repository root: `railway.toml`

Purpose: Tell Railway how to build and start **only** the payment server, avoiding `npm run build` for the CRA app on the same repo.

Typical contents:

```
[build]
builder = "NIXPACKS"
buildCommand = "npm install"

[deploy]
startCommand = "npm run payment-api"
healthcheckPath = "/health"
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 10
```

If Railway ignores this file, clear any **custom** Build/Start commands in the Railway UI so the file takes effect, then **Redeploy**.

11. Troubleshooting

Symptom	Likely cause	What to do
Neither <code>apiKey</code> nor <code>config.authenticator</code> provided	Missing <code>STRIPE_SECRET_KEY</code> on Railway or in <code>server/.env</code> locally	Set variable and restart / redeploy
CORS error in browser	Frontend origin not allowed	Add Vercel URL to <code>ALLOWED_ORIGINS</code> ; set <code>STRIPE_CORS_STRICT=1</code> only after origins are correct
Unexpected token <code>'<'</code> / HTML instead of JSON	Wrong <code>REACT_APP_PAYMENT_API_URL</code> or API down	Fix URL to <code>https://.../api/payments</code> ; check <code>/health</code>
Build fails on Railway with <code>npm run build</code>	<code>railway.toml</code> not used or overridden	Remove custom build command in UI; ensure <code>railway.toml</code> is on <code>main</code>
Payments work locally but not on Vercel	<code>REACT_APP_PAYMENT_API_URL</code> still localhost or missing	Set production URL in Vercel and redeploy

Symptom	Likely cause	What to do
ESLint / <code>no-unused-vars</code> fails CI	Full CRA build running on Railway	Use <code>railway.toml</code> as above

12. Exporting this guide to PDF

Source file: `docs/SKILLBRIDGE_DEPLOYMENT_GUIDE.md`.

Recommended — regenerate HTML + PDF from the repo

From the repository root:

```
npm install
npm run docs:deployment-html
```

This writes `docs/SkillBridge-Deployment-Guide.html` (open in Chrome/Edge → **Ctrl+P** → **Save as PDF**).

If you have **Microsoft Edge** installed, you can also generate a PDF from the command line (Windows):

```
& "${env:ProgramFiles(x86)}\Microsoft\Edge\Application\msedge.exe" --headless --disable
```

The repository may include a pre-built `docs/SkillBridge-Deployment-Guide.pdf` for convenience; re-run the steps above after editing the Markdown.

Other options

- **Pandoc:** `pandoc docs/SKILLBRIDGE_DEPLOYMENT_GUIDE.md -o SkillBridge-Deployment-Guide.pdf` (if Pandoc is installed).
- **VS Code / Cursor:** Open the `.md` or `.html` → Print → Save as PDF.

Document control

- **Audience:** developers and deployers of SkillBridge.

- **Secrets:** never embed real keys in this document; use Stripe and Supabase dashboards.
 - **Updates:** when adding new `REACT_APP_*` or server env vars, update `.env.example` / `server/.env.example` and this guide.
-

End of SkillBridge Deployment Guide

Create a PDF: Press Ctrl+P (Windows) or Cmd+P (Mac) → choose *Save as PDF* or *Microsoft Print to PDF* → Save.